

南开大学
Nankai University

南　　开　　大　　学

计　算　机　学　院

机器学习及应用实验报告

---

lab2——白葡萄酒质量回归

王 烨

年级：2024 级

专业：计算机科学与技术

指导教师：卫金茂

2026 年 10 月 20 日

目录

一、 实验目的 **1**

二、 数据集预处理及划分 **1**
（一） 数据预处理 1
（二） 数据划分 2

三、 实验环境 **2**

四、 实验思路 **3**
（一） 实验原理 3
1. 线性回归 3
2. 梯度下降 3
3. 岭回归 4
4. 特征相关性分析 4
5. 性能评价指标 4
（二） 基本要求：线性回归模型构建 4
（三） 中级要求：优化方法比较和特征分析 6
（四） 高级要求：岭回归模型构建 7

五、 实验结果解释及分析 **7**
（一） 学习率扫描和选择 8
（二） BGD 与 SGD 的对比分析 9
（三） 特征影响分析 10
（四） 特征相关性实验 11
（五） 岭回归中 λ 对 MSE 的影响 12
（六） 岭回归中 λ 对系数的影响 13
（七） 验证集选择 λ 的结果 14
（八） 各模型 MSE 测试汇总 15

六、 实验总结 **15**

一、 实验目的

- 使用白葡萄酒的理化指标建立线性回归模型，预测葡萄酒质量评分；
- 通过批量梯度下降和随机梯度下降优化模型，并比较学习率和正则化对预测效果的影响。

本人实验的所有代码，实验结果，包括但不限于可视化和算法脚本等都可以在 github 仓库 https://github.com/wangyeG2/Machine-learning-and-its-application中查看。

二、 数据集预处理及划分

本次实验选用的数据集为 UCI Machine Learning Repository 的 Wine Quality 数据集，文件名为 winequality-white.csv。

该数据集为 CSV 文本文件，包含 4898 份样本，共 12 列，分别为 11 个理化指标和 1 个质量评分。字段之间以分号隔开，每行是一份白葡萄酒样本，首行为字段名。

输入特征为前 11 列，目标变量为 quality，为离散整数评分，范围为 3–9，是本实验的数值型回归目标。

(一) 数据预处理

在对数据集进行分析时，我们发现，数据集内部出现了大量的重复样本，共计 937 条。

在本实验中，约定训练集和测试集的划分比例为 4：1。由于 4898 不能被 5 整除，取 $n_{\text{test}} = \lceil 0.2 \times 4898 \rceil = 980$。由概率计算可知，在完全随机划分下：

$$n_{\text{train}} = 3918, \qquad n_{\text{test}} = 980. \tag{1}$$

固定一对完全重复的行 (A, B)，A 落入训练集的概率为 $\dfrac{n_{\text{train}}}{N}$，此后 B 落入测试集的概率为 $\dfrac{n_{\text{test}}}{N-1}$；两种对称情形相加，得

$$P = 2 \times \frac{n_{\text{train}}}{N} \times \frac{n_{\text{test}}}{N-1} = \frac{2\, n_{\text{train}}\, n_{\text{test}}}{N\,(N-1)} \tag{2}$$

$$P = \frac{2 \times 3918 \times 980}{4898 \times 4897} = \frac{7\,679\,280}{23\,985\,506} \approx 32.02\%. \tag{3}$$

即任一对完全重复的行约有 32.02% 的概率分别落入训练集和测试集。

这是我们不能接受的，因为当重复数据对分别落入训练集和测试集时，会造成严重的数据泄露。在机器学习实验中，为了构建稳健、可泛化的机器学习模型，必须尽可能保证“两份数据相互独立、测试样本在训练时从未出现”的基本原则，否则会导致测试指标虚高，泛化能力错误评估。

所以在对数据集进行划分前，我们先执行去重操作：

```
df = pd.read_csv(DATA_FILE, sep=';')
print(f'原始数据: {df.shape}')
n_dup = int(df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)       # 去除完全重复样本
print(f'剔除完全重复样本 {n_dup} 条，剩余 {len(df)} 条')
```

作为消融对照，我们统计了若不去重直接按 4:1 分层划分的后果：在相同随机种子下，原始测试集 980 条中有 272 条在训练集中出现过完全相同的副本，与前述 32.02% 的理论概率估计量级一致，这些样本的评估已被污染，验证了去重的必要性。

此外，我们对去重后的数据集进行了排查，发现没有出现“特征完全相同，quality 不同”的样本，这避免了数据噪声。

(二) 数据划分

对去重后的数据，按照实验约定，以 4：1 的比例划分训练集和测试集。划分遵从分层抽样，按照 quality 的取值各自独立地按 4:1 抽样，再拼合成训练集与测试集：

```
# 分层 4:1 划分，保持各评分占比一致
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED)
diff = (pd.Series(y_train).value_counts(normalize=True).sort_index()
        - pd.Series(y_test).value_counts(normalize=True).sort_index()).abs()
print(f'训练/测试各评分占比最大绝对差 = {diff.max():.4f}')
```

需要说明的是，实验约定中的 4:1 划分指训练用途部分与测试部分的分割；为满足超参数选择不接触测试集的原则，我们在训练部分内部再按 4:1 分层划分子训练集与验证集。验证集仅用于学习率与后续正则化强度 λ 的选择，测试集仅在最终评估中出现一次，选定超参后模型在全部 3168 条训练数据上重训——最终模型仍使用了全部训练数据，且基本要求中的 4:1 训练/测试划分原样保留，二者并不矛盾。验证集划分如下：

```
Xs_tr, Xs_va, ys_tr, ys_va = train_test_split(
    X_train, y_train, test_size=0.2, stratify=y_train, random_state=0)
sc_lr = StandardScaler().fit(Xs_tr)   # 标准化参数只来自子训练集
Xl_tr = np.hstack([np.ones((len(Xs_tr), 1)), sc_lr.transform(Xs_tr)])
Xl_va = np.hstack([np.ones((len(Xs_va), 1)), sc_lr.transform(Xs_va)])
```

随后，我们对数据样本进行标准化处理，其变换参数完全由训练集确定，并根据实验约定，将偏置项并入特征向量：

```
scaler = StandardScaler().fit(X_train)   # 参数只由训练集确定
Xtr = np.hstack([np.ones((len(X_train), 1)), scaler.transform(X_train)])
Xte = np.hstack([np.ones((len(X_test), 1)), scaler.transform(X_test)])
```

三、 实验环境

实验环境搭建如下表所示：

表 1: 实验环境

配置项	基本要求	中级要求	高级要求
虚拟环境	.venv	.venv	.venv-dl
Python 版本	3.13.5	3.13.5	3.13.5
pip 库	numpy、pandas、scikit-learn、matplotlib	numpy、pandas、matplotlib	numpy、pandas、scikit-learn、matplotlib

四、 实验思路

(一) 实验原理

1. 线性回归

线性回归是监督学习中最基础的回归算法，其基本思想是：假设目标变量与各特征之间存在线性关系，通过在训练集上最小化预测值与真实值之间的平方误差来学习模型参数。

设训练集为 $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$，其中 $\boldsymbol{x}^{(i)} \in \mathbb{R}^p$ 为样本特征向量，$y^{(i)}$ 为连续型目标值。将偏置项并入特征向量，即 $\boldsymbol{x} \leftarrow [1, x_1, \ldots, x_p]^{\mathrm{T}}$，模型的假设函数为

$$h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{x}^{\mathrm{T}}\boldsymbol{\theta} \tag{4}$$

模型以均方误差（Mean Squared Error, MSE）作为损失函数：

$$MSE(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}\big(h_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}) - y^{(i)}\big)^2 \tag{5}$$

2. 梯度下降

梯度下降是求解线性回归的迭代优化算法，其基本思想是：沿损失函数的负梯度方向迭代更新参数，直至收敛。损失函数对参数的梯度为

$$\nabla MSE(\boldsymbol{\theta}) = \frac{2}{m}X^{\mathrm{T}}\big(X\boldsymbol{\theta} - \boldsymbol{y}\big) \tag{6}$$

本实验中，梯度下降采用两种方法：

- 批量梯度下降（Batch Gradient Descent, BGD）：每轮迭代使用全部 m 个训练样本估计梯度，更新规则为：
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\,\frac{2}{m}X^{\mathrm{T}}\big(X\boldsymbol{\theta} - \boldsymbol{y}\big) \tag{7}$$
其梯度方向准确，收敛曲线平滑，但单轮计算代价与样本总量成正比。

- 随机梯度下降（Stochastic Gradient Descent, SGD）。每轮迭代将训练集随机打乱后逐样本更新，更新规则为：
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - 2\alpha\,\boldsymbol{x}^{(i)}\big(\boldsymbol{x}^{(i)\mathrm{T}}\boldsymbol{\theta} - y^{(i)}\big) \tag{8}$$
其中 $(\boldsymbol{x}^{(i)}, y^{(i)})$ 为当前遍历到的样本。

  单次更新仅依赖一个样本，计算代价低，但更新方向含随机噪声，收敛曲线呈现明显震荡。

在本实验中，学习率 α 会较为显著地影响梯度下降的性能：α 过大时损失震荡甚至发散，α 过小时收敛缓慢、需更多训练轮次。此外，BGD 与 SGD 的差异体现在误差、收敛速度与稳定性三个维度上，实验将在相同轮数下通过收敛曲线进行定量比较。

3. 岭回归

当特征之间存在高度线性相关（多重共线性）时，$X^{\mathrm{T}}X$ 接近奇异矩阵，最小二乘解的系数会在正负之间剧烈摆动，模型不稳定。岭回归在 MSE 损失上加入 L2 正则项以约束参数规模：

$$\mathcal{J}(\boldsymbol{\theta}) = \frac{1}{m}\|X\boldsymbol{\theta} - \boldsymbol{y}\|_2^2 + \lambda \sum_{j=1}^{p} \theta_j^2 \tag{9}$$

其中 $\lambda \geq 0$ 为正则化强度，偏置项 θ_0 不参与正则化。令 $\nabla \mathcal{J}(\boldsymbol{\theta}) = \mathbf{0}$ 可得解析解：

$$\boldsymbol{\theta} = \left(X^{\mathrm{T}}X + m\lambda I'\right)^{-1} X^{\mathrm{T}}\boldsymbol{y} \tag{10}$$

其中 $I' = \mathrm{diag}(0, 1, \ldots, 1)$，用于豁免偏置项。由于 $X^{\mathrm{T}}X + m\lambda I'$ 对任意 $\lambda > 0$ 恒为正定矩阵，解总是存在且唯一。岭回归亦可用批量梯度下降求解，其更新规则仅在线性回归的梯度上追加正则项梯度：

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\left[\frac{2}{m}X^{\mathrm{T}}\left(X\boldsymbol{\theta} - \boldsymbol{y}\right) + 2\lambda I'\boldsymbol{\theta}\right] \tag{11}$$

在本实验中，正则化强度 λ 会较为显著地影响岭回归的性能：λ 过小时正则化近乎失效，共线特征的系数依旧不稳定；λ 过大时系数被过度压缩，模型欠拟合。测试 MSE 随 λ 增大通常呈现先降后升的 U 形曲线，且随着 λ 增大，同一特征在不同数据划分下系数的波动显著收窄。

4. 特征相关性分析

Pearson 相关系数衡量两个特征之间线性相关的方向与强度：

$$r_{jk} = \frac{\sum_{i=1}^{m}\left(x_j^{(i)} - \bar{x}_j\right)\left(x_k^{(i)} - \bar{x}_k\right)}{\sqrt{\sum_{i=1}^{m}\left(x_j^{(i)} - \bar{x}_j\right)^2}\sqrt{\sum_{i=1}^{m}\left(x_k^{(i)} - \bar{x}_k\right)^2}} \tag{12}$$

其中 $\bar{x}_j$、$\bar{x}_k$ 为对应特征的均值，$|r_{jk}|$ 越接近 1 表明线性相关性越强。在本实验中，特征经标准化后方差一致，回归系数的绝对值可直接横向比较，用于排序特征的重要性；进一步对某一特征注入一个标准差的高斯噪声并度量预测值的平均变化幅度，可从敏感性的角度交叉验证该特征的影响程度。

5. 性能评价指标

- **训练误差与测试误差**：模型在训练集与测试集上的 MSE。前者反映拟合能力，后者反映泛化能力，两者的差距是判断过拟合与欠拟合的依据。
- **收敛曲线**：训练/测试 MSE 随训练轮次的变化轨迹，用于比较不同优化方法与不同学习率下的收敛速度与稳定性。

(二) 基本要求：线性回归模型构建

实验约定，不采用 sklearn 库的相关函数实现线性回归的核心内容，我们按照实验原理手动实现：

```
def mse(X, y, th):
    v = float(np.mean((X @ th - y) ** 2))
    return v if np.isfinite(v) else np.inf

def bgd(X, y, Xte, yte, lr, n_epochs=N_EPOCHS):
```

```
    th = np.zeros(X.shape[1])
    h_tr, h_te = [mse(X, y, th)], [mse(Xte, yte, th)]
    for _ in range(n_epochs):
        th -= lr * (2.0 / len(y)) * X.T @ (X @ th - y)
        v = mse(X, y, th)
        if not np.isfinite(v):                    # 发散
            h_tr.append(np.inf); h_te.append(np.inf)
            break
        h_tr.append(v); h_te.append(mse(Xte, yte, th))
    return th, h_tr, h_te

def sgd(X, y, Xte, yte, lr, n_epochs=SGD_EPOCHS, record='epoch'):
    th = np.zeros(X.shape[1])
    h_tr, h_te = [mse(X, y, th)], [mse(Xte, yte, th)]
    rng = np.random.default_rng(SEED)
    for _ in range(n_epochs):
        for i in rng.permutation(len(y)):
            xi, yi = X[i], y[i]
            th -= lr * 2.0 * xi * (xi @ th - yi)
            if record == 'update':
                h_tr.append(mse(X, y, th)); h_te.append(mse(Xte, yte, th))
        if record == 'epoch':
            h_tr.append(mse(X, y, th)); h_te.append(mse(Xte, yte, th))
            if not np.isfinite(h_tr[-1]):         # 发散
                break
    return th, h_tr, h_te
```

BGD 和 SGD 函数都在每个 epoch 记录一次训练/评估集 MSE，即收敛曲线的数据来源，并检测发散情况。函数中第 3、4 个参数（Xte/yte）为通用评估集：扫描选型阶段传入验证集，最终训练阶段传入测试集。

随后设置学习率，由于 SGD 的方差较大，需要的学习率相较于 BGD 更小。逐学习率在验证集上训练后，绘制相关曲线，然后选择最佳学习率：

```
BGD_LRS      = [0.01, 0.05, 0.1, 0.5]             # BGD 候选学习率
SGD_LRS      = [0.001, 0.005, 0.01, 0.05, 0.1]    # SGD 候选学习率

# 学习率选定
def analyze_lr(name, res):
    print(f'\n===== {name} 不同学习率分析（验证集协议） =====')
    print(f'{"lr":>8} | {"最终训练MSE":>12} | {"最终验证MSE":>12} '
          f'| {"最低验证MSE":>12} | 结果')
    best_lr, best_fin = None, np.inf
    for lr, (h_tr, h_va) in res.items():
        va = np.asarray(h_va, float)
        if not np.isfinite(va[-1]) or va[-1] > DIVERGE_TOL * va[0]:
            print(f'{lr:>8} | {"—":>12} | {"—":>12} | {"—":>12} | 发散')
            continue
        print(f'{lr:>8} | {float(h_tr[-1]):>12.4f} | {va[-1]:>12.4f} '
              f'| {va.min():>12.4f} | 收敛')
```

```
        if va[-1] < best_fin:
            best_lr, best_fin = lr, float(va[-1])
    print(f'>>> {name} 选定 lr={best_lr}（验证MSE最低 {best_fin:.4f}）')
    return best_lr

# 扫描传入验证集，测试集不参与选型
bgd_res = {lr: bgd(Xl_tr, ys_tr, Xl_va, ys_va, lr)[1:] for lr in BGD_LRS}
sgd_res = {lr: sgd(Xl_tr, ys_tr, Xl_va, ys_va, lr)[1:] for lr in SGD_LRS}
lr_b = analyze_lr('BGD', bgd_res)
lr_s = analyze_lr('SGD', sgd_res)
```

(三) 中级要求：优化方法比较和特征分析

在这一阶段，我们首先需要在同一环境下比较 BGD 和 SGD 的差异，包括误差水平、收敛速度和波动性等。为了公平对比，我们规定训练轮数 epoch 均为 300，然后量化相关指标对比分析，并绘制收敛曲线对比。限于篇幅，此处不展示具体实现，执行结果在“实验结果解释及分析”一节展示。

随后，我们需要对相关特征进行分析。此处为了避免近似误差，我们使用精确 OLS 解求出模型系数，并列出系数表，从中选取最大的 density，residual sugar，alcohol 三个特征进行分析，方法为注入一个标准差的高斯噪声，度量预测变化：

```
coef_s = pd.Series(th_ols[1:], index=feat_names)
print('\n(a) 标准化模型系数:')
print(coef_s.sort_values(key=np.abs, ascending=False).to_string(float_format=
    '%.4f'))

top3 = coef_s.abs().nlargest(3).index.tolist()
print(f'\n选取特征: {top3}')
print('\n(b) 扰动实验')
rng = np.random.default_rng(0)
pred0 = Xte @ th_ols
avgs = []
for nm in top3:
    Xp = Xte.copy()
    Xp[:, feat_names.index(nm) + 1] += rng.normal(0, 1, len(Xp))
    avgs.append(float(np.mean(np.abs(Xp @ th_ols - pred0))))
    print(f'  {nm:<24} 系数={coef_s[nm]:+.4f} 平均|Δ预测|={avgs[-1]:.4f}')
```

随后，我们需要根据训练集的 Pearson 相关系数选出最相关的一对特征，将其标准化后，比较保留两者和移除其中一个特征时的模型系数与测试集 MSE，分析相关特征对系数稳定性和预测效果的影响：

```
corr = pd.DataFrame(X_train, columns=feat_names).corr()
c_abs = corr.abs().mask(np.eye(len(feat_names), dtype=bool))   # 排除对角线
f1, f2 = c_abs.stack().idxmax()

schemes = {'全模型(保留两者)': list(range(12)),
           f'移除{f2}':     [c for c in range(12) if c != c2],
```

```
            f'仅用{f1}+{f2}': [0, c1, c2],
            f'仅用{f1}':      [0, c1]}
```

（四） 高级要求：岭回归模型构建

在这一阶段，我们需要手动实现岭回归的核心部分，并在训练样本上进行求解。[1] 此处我们采用两种求解方法，分别为解析法和梯度下降法，两者形成对照：

```
# 解析法
def ridge_analytic(X, y, lam):
    n = X.shape[1]
    I_reg = np.eye(n); I_reg[0, 0] = 0.0
    return np.linalg.solve(X.T @ X + X.shape[0] * lam * I_reg, X.T @ y)

# 梯度下降法
def ridge_bgd(X, y, lam, lr, n_epochs=10000, tol=1e-12):
    th = np.zeros(X.shape[1])
    reg = np.ones_like(th); reg[0] = 0.0
    for _ in range(n_epochs):
        delta = lr * ((2.0 / len(y)) * X.T @ (X @ th - y) + 2.0 * lam * reg *
            th)
        th -= delta
        if np.max(np.abs(delta)) < tol:
            break
    return th
```

随后对 λ 表内的 6 个值进行扫描，使用解析法分别求解，随后选定一个 λ 值，使用梯度下降法求解，形成对照：

```
rows = []
for lam in LAMS:
    th = ridge_analytic(Xtr, y_train, lam) #解析法求解
    rows.append([lam, float(np.mean((Xtr @ th - y_train) ** 2)),
                 float(np.mean((Xte @ th - y_test) ** 2)), th[c1], th[c2]])

# 对照实验
lam_demo = 1e-2
th_a = ridge_analytic(Xtr, y_train, lam_demo)
th_g = ridge_bgd(Xtr, y_train, lam_demo, lr=0.1)
ridge_report(f'BGD lambda={lam_demo:g}', th_g, Xtr, y_train, Xte, y_test)
print(f'解析法 vs BGD 系数最大差 = {np.max(np.abs(th_a - th_g)):.2e}')
```

随后绘制 λ 与 MSE 的关系曲线，以及 λ 系数变化路径图，展示 L2 正则化的稳定作用；此外，再次引入与 λ 相关的验证实验，此处不再赘述。

五、 实验结果解释及分析

完整的输出结果被重定向至 output.txt 中，此处展开分析。

[1] 声明：对于岭回归的求解，偏置项 θ_0 不参与惩罚。

(一) 学习率扫描和选择

表 2: BGD 不同学习率下的 MSE（验证集协议，300 epoch）

学习率	最终训练 MSE	最终验证 MSE	最低验证 MSE	结果
0.01	0.5622	0.5369	0.5369	收敛
0.05	0.5595	0.5326	0.5326	收敛
0.10	0.5584	**0.5303**	**0.5303**	收敛
0.50	—	—	—	发散

表 3: SGD 不同学习率下的 MSE（验证集协议，100 epoch）

学习率	最终训练 MSE	最终验证 MSE	最低验证 MSE	结果
0.001	0.5664	**0.5350**	0.5257	收敛
0.005	0.6152	0.5994	0.5341	收敛
0.010	0.6738	0.6713	0.5428	收敛
0.050	1.6860	1.6568	0.7493	收敛（震荡）
0.100	—	—	—	发散

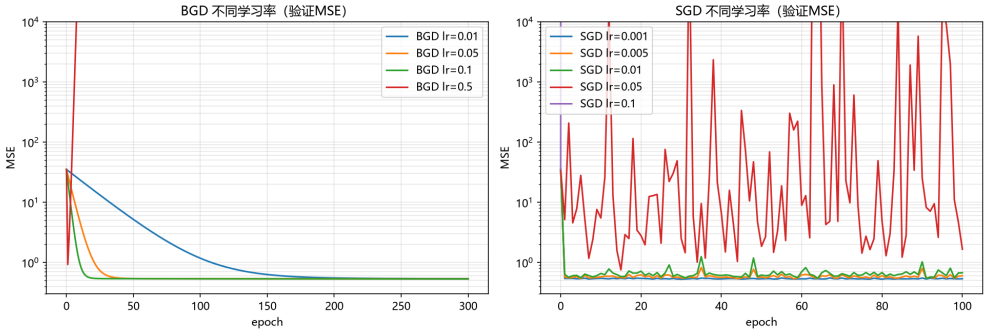


图 1: 不同学习率下的 MSE 收敛曲线

根据实验结果可以得出以下结论:

- BGD 对学习率较不敏感：三个收敛学习率的验证 MSE 极差仅 0.0066，小于验证集 MSE 的标准误差（$\sigma^2 \approx 0.55$、$n \approx 634$），统计上可认为 0.01–0.1 之间平手；其中 lr=0.1 收敛最快，据验证协议选定 **lr=0.1**；lr=0.5 发散。
- SGD 对学习率高度敏感：随着 lr 增大，SGD 的震荡幅度增大，最终验证 MSE 单调上升；lr=0.05 严重震荡，lr=0.1 直接发散。据验证协议选定 **lr=0.001**。这本身是因为 SGD 单样本梯度更新所带来的大方差导致的。
- 学习率与 MSE 的 trade-off: lr 较小时，MSE 的收敛速度慢，迭代轮数多；lr 过大时，MSE 会趋向震荡甚至发散。

(二) BGD 与 SGD 的对比分析

表 4: BGD 与 SGD 比较（300 epoch）

方法	学习率	最终训练 MSE	最终测试 MSE	训练 MSE 进入终值 ×1.05 所需 epoch
BGD	0.1	0.5521	**0.5697**	17
SGD	0.001	0.5578	0.5842	**1**

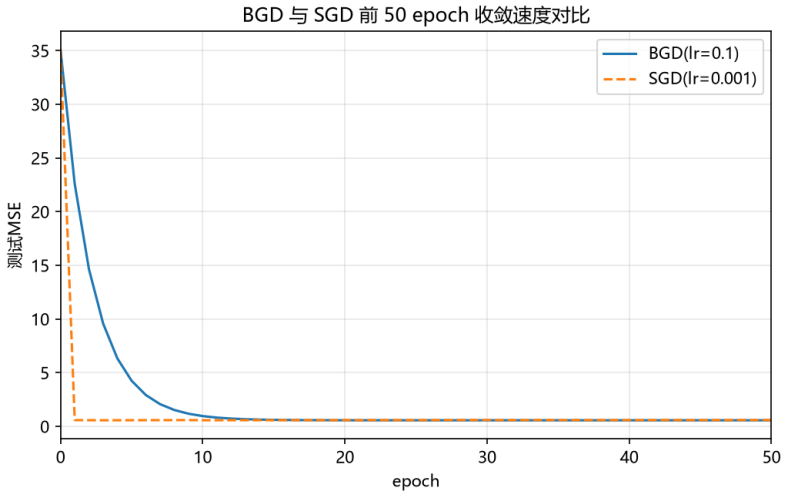


图 2: BGD 和 SGD 的收敛对比曲线

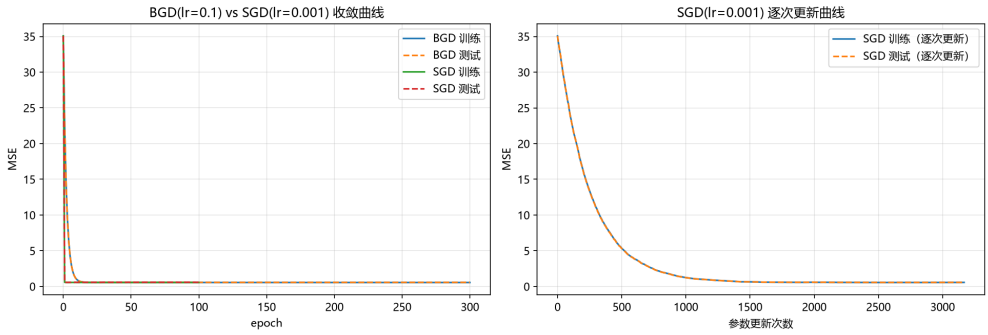


图 3: BGD 与 SGD 的全程收敛曲线（左）与 SGD 逐次参数更新曲线（右）

由上述实验结果可知，在相同训练轮数和当前数据集情况下，BGD 的最终误差准确性更高，这是因为 BGD 是确定性下降，随着训练轮数增加呈现逐步逼近最终到达的趋势，而 SGD 的单样本更新携带随机噪声，参数在最优值附近长期小幅漂移。SGD 300 epoch 的最终测试 MSE 略高于扫描时 100 epoch 的结果，即为这种噪声漂移的体现。

而 SGD 的收敛速度远快于 BGD：SGD 仅 1 个 epoch 即进入最终训练 MSE 的 1.05 倍邻域，BGD 需要 17 个 epoch。这是因为 SGD 每个 epoch 内对每个样本各执行一次参数更新，相同 epoch 数下参数更新次数约为 BGD 的 m 倍（m 为训练样本量），单位计算量对梯度信息的利用更充分；代价是更新方向含随机噪声，收敛曲线伴随震荡。

上述结论区分了 BGD 和 SGD 的各自优势，即 BGD 平滑单调，稳定性强，而 SGD 收敛速度快，但存在噪声震荡，稳定性差。

(三) 特征影响分析

表 5: 单特征注入 1σ 高斯噪声后的预测变化

特征	标准化系数	平均 $\lvert\Delta$预测$\rvert$
density	-0.5599	0.4499
residual sugar	$+0.4313$	0.3329
alcohol	$+0.1685$	0.1409

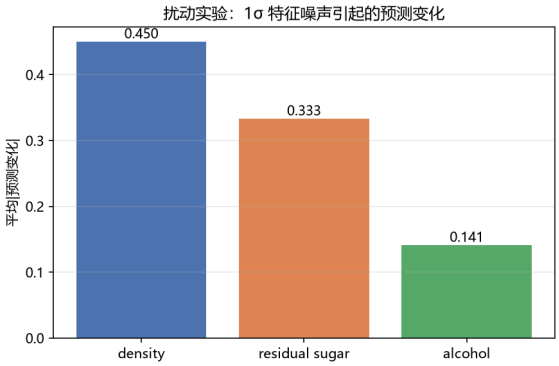


图 4: 扰动实验结果

在标准化空间中，对测试集每个样本的第 j 个特征独立注入一个标准差的高斯噪声，其余特征保持不变：

$$x'_{ij} = x_{ij} + \delta_i, \qquad \delta_i \sim \mathcal{N}(0,1), \quad i = 1, \ldots, n_{\text{te}}, \tag{13}$$

并考察预测变化 $\Delta h^{(i)} = h(\boldsymbol{x}'_i) - h(\boldsymbol{x}_i)$，以平均绝对变化 $\overline{|\Delta h|} = \frac{1}{n_{\text{te}}} \sum_i |\Delta h^{(i)}|$ 作为该特征的预测敏感度度量。由于特征已标准化，一个标准差在原始量纲下恰为该特征自身的典型波动幅度，因此本实验度量的是：当该特征的测量出现与自身波动同量级的误差时，模型预测会移动多远。

由实验结果可得出以下结论：

- 标准化和扰动预测相互印证：根据数学预期，$|\Delta$预测$|$ 应当为标准化系数绝对值的 0.8 倍左右，测试结果符合这一期望，且扰动实验的敏感度排序结果和标准化系数排序完全一致。
- 特征值对质量的影响：density 对质量评分影响最大且为负，alcohol 影响为正。物理上，糖分高、酒精度低都会推高密度，密度低的酒往往发酵更充分，而酒精度更高的葡萄酒通常评分更高，模型得出的结论与之一致。此处没有提及 residual sugar，原因见下述“特征相关性实验”。

（四） 特征相关性实验

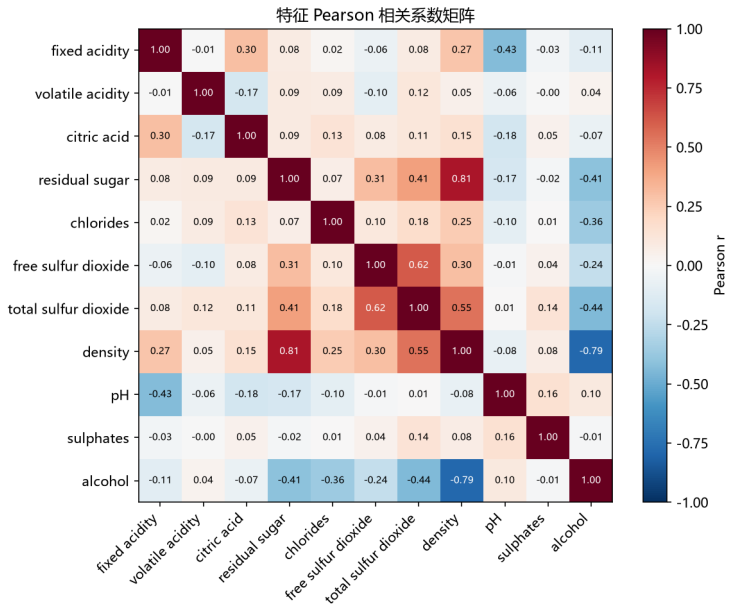


图 5: 特征 Pearson 相关系数矩阵热力图

根据特征 Pearson 相关系数矩阵，最相关特征对为 residual sugar↔density，r=0.8130，这与物理规律一致，即糖分溶解于酒体后使密度增大。

表 6: 最相关特征对各方案对比

方案	训练 MSE	测试 MSE	residual sugar 系数	density 系数
全模型（保留两者）	0.5512	0.5788	+0.4313	−0.5599
移除 density	0.5601	0.5726	+0.0960	—
仅用 residual sugar + density	0.6315	0.6792	+0.4352	−0.6659
仅用 residual sugar	0.7818	0.7838	−0.1062	—

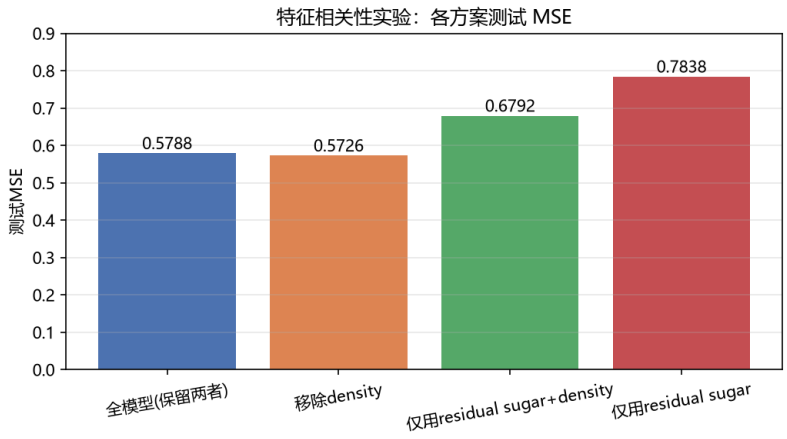


图 6: 相关特征对 MSE 的影响

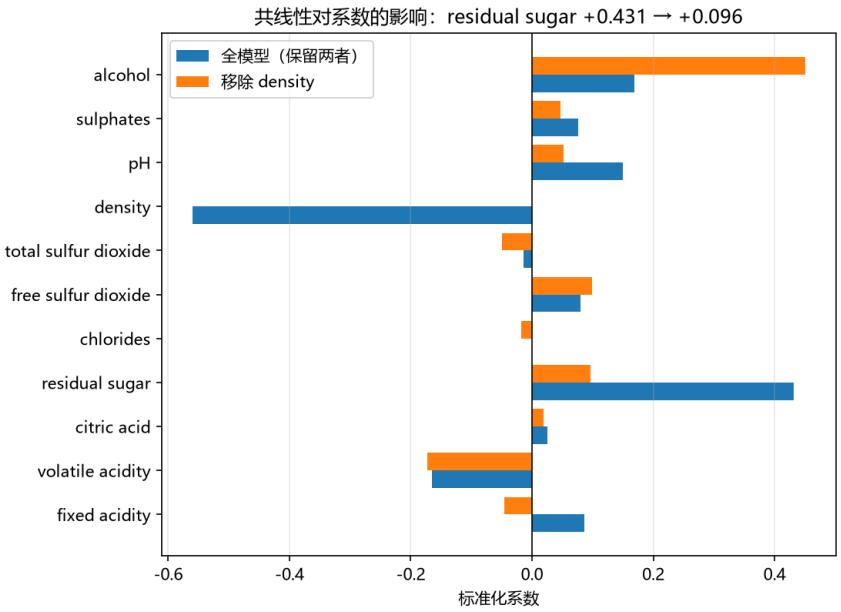


图 7: 共线性对系数的影响

而对最相关特征对进行特征相关性实验时，结合上述图表，可得出如下结论：

- 共线性破坏了系数稳定性：一旦移除 density，residual sugar 的系数骤降至 0.0960，这说明在全模型中，residual sugar 的正向影响实际与 density 共享，单系数不可信。而如果只启用 residual sugar，系数出现了符号反转，系数出现了不可解释性。此外，移出 density 会导致其他系数连锁变化，如 alcohol 上升至 0.47，说明强相关特征对的个别系数不具稳定性。
- 共线性有利于预测结果：移出 density 对于 MSE 几乎没有影响，这是因为共线特征对预测的贡献高度冗余，去除共线项相当于去除冗余。但是如果只使用 density 和 residual sugar，MSE 骤然上升，只启用 residual sugar 时，MSE 继续增大，这也说明了强相关特征对不可置信，需要其他增量信息配合。

（五） 岭回归中 λ 对 MSE 的影响

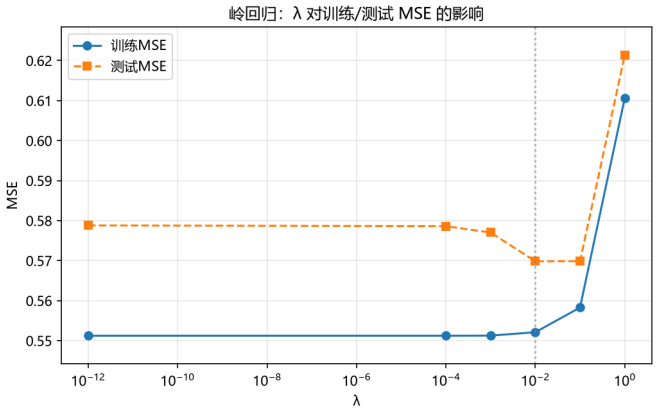


图 8: λ 对 MSE 的影响曲线

可以发现，随着 λ 增大，训练 MSE 单调上升，而测试 MSE 出现了明显的“U”型曲线，在 0.01 附近取最小值。这是符合预期的，因为岭回归目标比 OLS 多出非负惩罚项，λ 越大对训练数据的拟合越受抑制。而对于测试 MSE，λ 过小时正则化不明显；增大到一定规模时，正则化的影响开始显现；而 λ 过大时，出现了过度正则化导致欠拟合。

需要说明的是，本节的 λ 扫描按实验要求展示各 λ 下测试 MSE 的变化，但 λ 的最终选定仅依据验证集协议；测试集扫描得到的最优 $\lambda = 0.01$ 仅作为诊断对照，不用于决策。

(六) 岭回归中 λ 对系数的影响

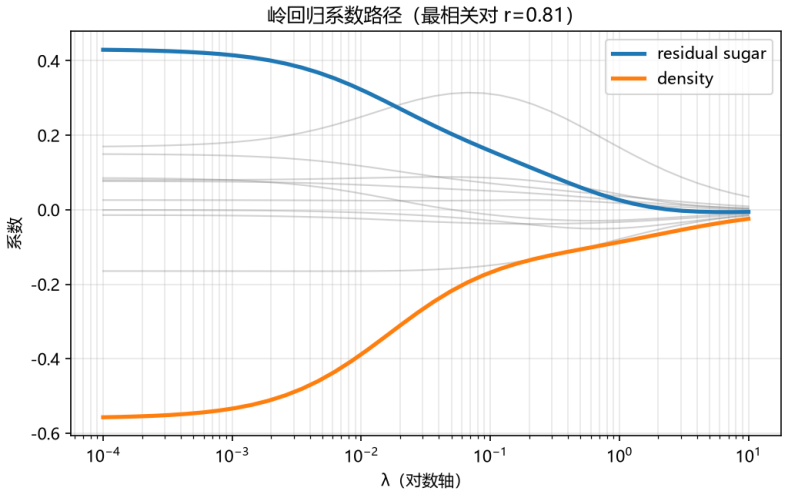


图 9: λ 对系数的路径曲线

随着 λ 增大，对于最相关特征对的两个系数 density 和 residual sugar，同步平滑地向 0 收缩，两者轨迹彼此靠拢，这是 L2 正则化的作用体现：将共线对共同解释的方差对称地分摊给两者并同步压缩。这是正则化与 OLS 的重要区分，因为后者把系数全部堆在其中之一，移除后便出现了先前所述的系数波动。

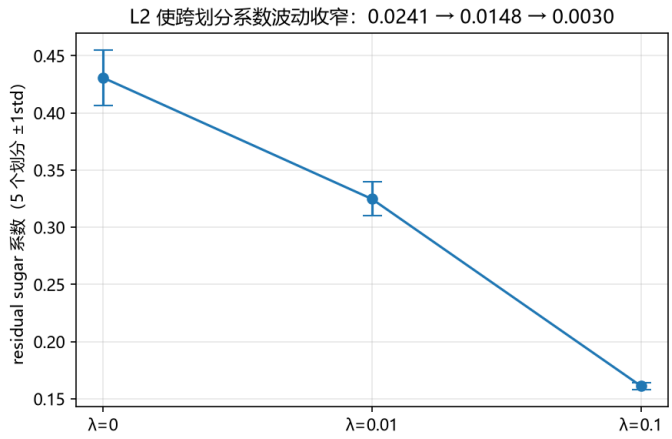


图 10: λ 对跨划分系数的影响曲线

表 7: 系数稳定性：5 个不同随机划分下 residual sugar 系数的波动

λ	系数均值	跨划分标准差	相对降幅
0	+0.4308	0.0241	—
0.01	+0.3247	0.0148	−38.6%
0.1	+0.1613	0.0030	−87.6%

而上述的图表更好地说明了 λ 对跨划分系数的影响，随着 λ 增大，波动范围逐步收窄，先前剧烈漂移的系数在 L2 正则化处理下得到明显抑制，代价是均值逐步下降，偏差逐步增大，这是方差与偏差的 trade-off。

上述实验结果证明了 L2 正则化的分摊机制既保留了特征的预测信息，又消除了系数膨胀。

（七） 验证集选择 λ 的结果

这是我们在“高级要求”部分实现时引入的对于 λ 的验证实验，这一实验的目的是：保持测试集的纯净性，避免数据泄露，保证对于测试集的 MSE 无偏，增加可信度。如果直接通过扫描 λ 选取最优 MSE 对应的值，实际是对测试集的复用，复用测试集会导致泛化性能估计出现过度乐观的偏差。

最终实验结果如下：

表 8: 验证集选 λ

λ	0	10^{-4}	10^{-3}	10^{-2}	10^{-1}	10^{0}
验证 MSE	**0.5278**	**0.5278**	0.5280	0.5302	0.5370	0.5914

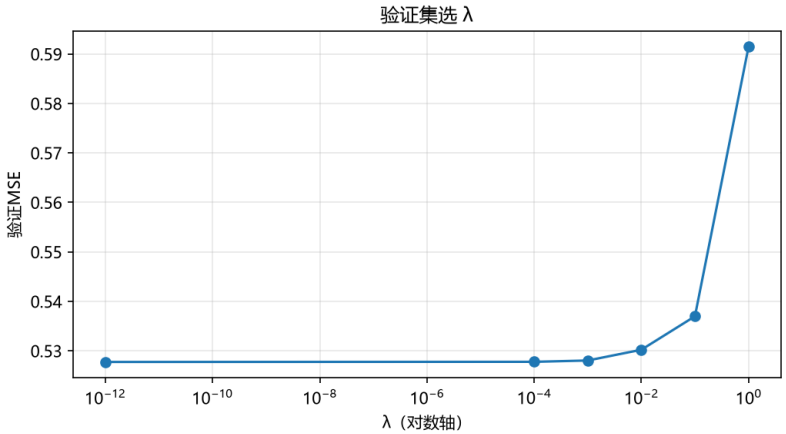


图 11: 验证集下 λ 对 MSE 的影响曲线

实验结果证明了我们先前的推论：通过验证集选出的 λ 和实际在测试集下执行的最优 λ 并不一致，且 MSE 随 λ 的变化曲线趋势也不一致，最终 MSE 也存在差异。这是诚实实验的结果。

注意到验证集选出的 λ=0，此时岭回归实际退化为 OLS。它说明在本数据规模和噪声水平下，L2 正则化的收益不足以从抽样噪声中被可靠地检测出来，在验证集协议下，OLS 是可选的最优决策。

该验证协议同样适用于先前学习率的选择: 学习率扫描在相同的子训练集/验证集上进行, 测试集不参与选型。修正协议后 BGD 的选型由 0.01 变为 0.1，直接量化了原协议约 0.003 的乐观偏差。

(八) 各模型 MSE 测试汇总

最后，我们汇总所有回归模型对测试集的 MSE，进行横向比较:

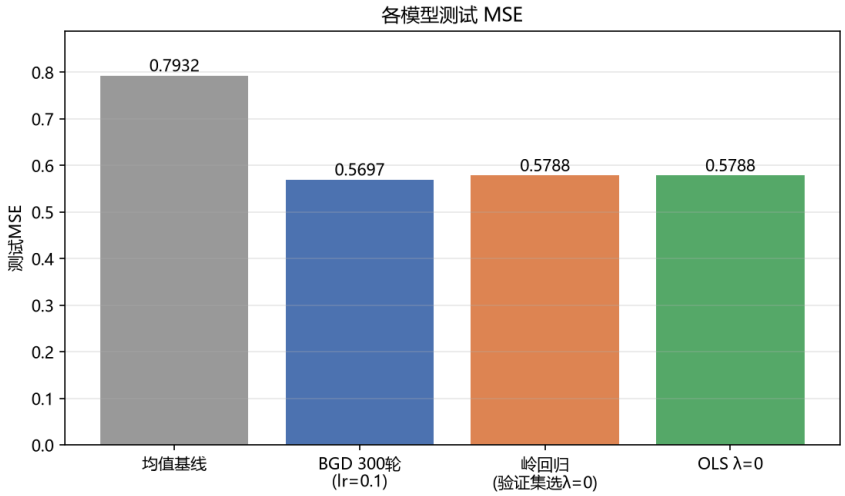


图 12: 各模型 MSE 测试汇总

可以得出以下结论:

- 模型本身价值远大于正则化收益：所有回归模型的测试 MSE 相较于 baseline 均明显下降，而不同模型间差异极小。
- 早停 BGD 与最优岭回归持平：BGD 在 300 epoch 处尚未完全收敛即停止，隐式地限制了模型复杂度，是一种隐式的容量约束。验证协议下其测试 MSE 为 0.5697，与测试集扫描最优的岭回归 $\lambda = 0.01$ 持平，均略优于 OLS。
- 验证集未选出正则化收益：验证协议选定的 $\lambda = 0$，图中“岭回归（验证集选 $\lambda = 0$）”与“OLS $\lambda = 0$”两柱数值相同，二者退化为同一模型。

六、 实验总结

本实验在白葡萄酒质量数据集上完整实现了线性回归的全流程：首先通过去重与分层划分建立无泄漏的评估基础，随后从零实现 BGD 与 SGD 两种优化方法，在验证集协议下完成学习率选择与收敛速度对比，继而通过系数排序、扰动实验与共线性实验完成特征影响分析，最后手写实现岭回归并利用解析法与梯度法交叉验证，完成 λ 扫描、验证集调参与系数稳定性分析。实验达到了掌握线性回归原理、梯度优化方法、正则化机制及规范模型评估流程的目的。